QCAA HW1

施子嚴

B12505009

Github: QCAA_HW1 (請按超連結)

Problem 1

Problem Setup

We consider the target function

$$f(x_1, x_2) = \sin(e^{x_1} + x_2)$$

The regression task is defined on two different domains:

- Training domain: $x_1, x_2 \in [0, 0.5]$
- Test domain: $x_1, x_2 \in [0.5, 1]$

The random seed is set to the numerical part of the student ID:

- Seed: `12505009`

The evaluation metric is mean squared error (MSE).

To search for a good data reuploading circuit, I performed a full sweep over:

- Number of qubits: `2, 3, 4`
- Number of layers: `2, 4, 6, 8`
- Encoding types: `rx_ry`, `ry_rz`
- Learning rate: `0.005`

All models were trained and tested using the full dataset size:

- `1000` training samples
- `1000` test samples

The best configuration found in the sweep is:

- `4 qubits`
- `8 layers`
- `rx_ry` encoding
- learning rate `0.005`

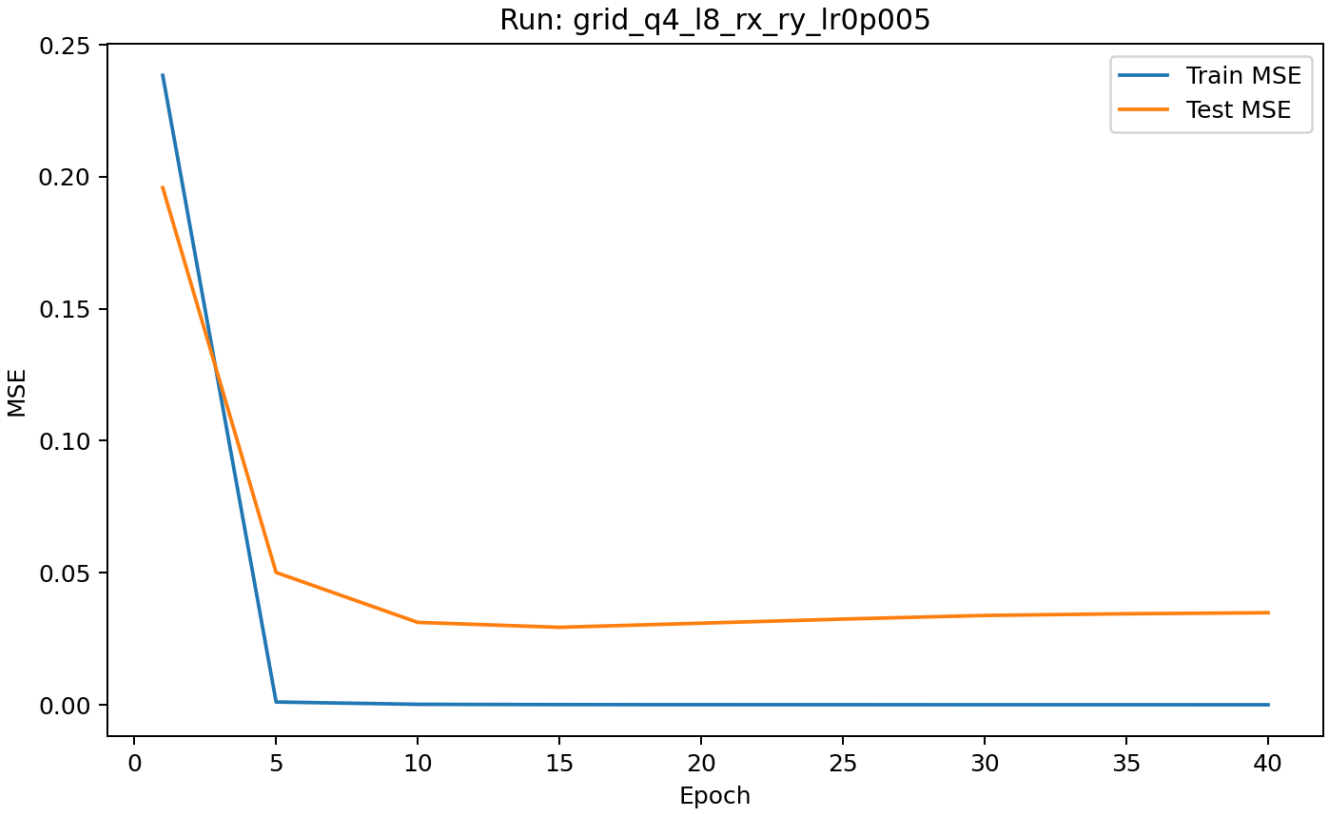
This model achieved:

- Best test MSE: `0.029371`
- Final test MSE at epoch `40`: `0.034888`

(a) Training Loss And Test Loss Versus Epoch

The best model is `grid_q4_l8_rx_ry_lr0p005`.

Best-model training curve:



Observation:

- The best model improves quickly during the early stage of training.
- The test MSE drops from `0.195855` at epoch `1` to `0.029371` at epoch `15`.
- After epoch `15`, the test MSE increases slightly, but the degradation is mild. At epoch `40`, the test MSE is still only `0.034888`.
- This suggests that the model is expressive enough to capture the target function while maintaining relatively stable generalization on the unseen test domain.

(b) Comparison Of Hyperparameter Configurations

The table below compares all tested configurations. It includes the number of qubits, the number of layers, the encoding type, the number of trainable parameters, the best test MSE observed during training, and the final test MSE at the last epoch.

Model	Qubits	Layers	Encoding	Parameters	Best Epoch	Best Test MSE	Final Test MSE	Train MSE
`grid_q4_l8_rx_ry_lr0p005`	4	8	`rx_ry`	103	15	0.029371	0.034888	0.000090
`grid_q4_l4_rx_ry_lr0p005`	4	4	`rx_ry`	55	40	0.043251	0.043251	0.000070
`grid_q4_l4_ry_rz_lr0p005`	4	4	`ry_rz`	55	10	0.045497	0.065842	0.000595
`grid_q4_l2_rx_ry_lr0p005`	4	2	`rx_ry`	31	1	0.053724	0.097634	0.064411
`grid_q3_l4_rx_ry_lr0p005`	3	4	`rx_ry`	42	40	0.068188	0.068188	0.000043
`grid_q4_l2_ry_rz_lr0p005`	4	2	`ry_rz`	31	1	0.066373	0.120815	0.010686
`grid_q2_l8_rx_ry_lr0p005`	2	8	`rx_ry`	53	40	0.070377	0.070377	0.000088
`grid_q2_l8_ry_rz_lr0p005`	2	8	`ry_rz`	53	40	0.073511	0.073511	0.000066

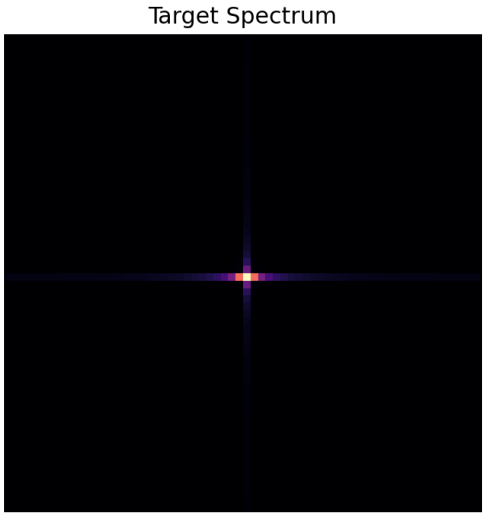
Model	Qubits	Layers	Encoding	Parameters	Best Epoch	Best Test MSE	Final Test MSE	Train MSE
`grid_q2_l2_rx_ry_lr0p005`	2	2	`rx_ry`	17	1	0.084751	0.496933	0.614499
`grid_q4_l8_ry_rz_lr0p005`	4	8	`ry_rz`	103	35	0.087337	0.088646	0.000051
`grid_q3_l6_ry_rz_lr0p005`	3	6	`ry_rz`	60	1	0.088204	0.141820	0.057232
`grid_q4_l6_ry_rz_lr0p005`	4	6	`ry_rz`	79	1	0.091493	0.337350	0.193960
`grid_q2_l2_ry_rz_lr0p005`	2	2	`ry_rz`	17	1	0.095744	0.420966	0.624392
`grid_q3_l8_ry_rz_lr0p005`	3	8	`ry_rz`	78	1	0.104570	0.206023	0.403502
`grid_q3_l6_rx_ry_lr0p005`	3	6	`rx_ry`	60	1	0.105683	0.247177	0.068129
`grid_q3_l8_rx_ry_lr0p005`	3	8	`rx_ry`	78	1	0.121496	0.431889	0.432931
`grid_q2_l4_ry_rz_lr0p005`	2	4	`ry_rz`	29	40	0.138817	0.138817	0.000044
`grid_q4_l6_rx_ry_lr0p005`	4	6	`rx_ry`	79	1	0.182120	0.465224	0.137424
`grid_q3_l2_ry_rz_lr0p005`	3	2	`ry_rz`	24	40	0.245351	0.245351	0.000069
`grid_q2_l6_ry_rz_lr0p005`	2	6	`ry_rz`	41	1	0.253586	0.554670	0.012097
`grid_q2_l6_rx_ry_lr0p005`	2	6	`rx_ry`	41	1	0.296365	0.757952	0.015331
`grid_q2_l4_rx_ry_lr0p005`	2	4	`rx_ry`	29	40	0.317951	0.317951	0.000040
`grid_q3_l4_ry_rz_lr0p005`	3	4	`ry_rz`	42	40	0.320085	0.320085	0.000027
`grid_q3_l2_rx_ry_lr0p005`	3	2	`rx_ry`	24	40	0.343280	0.343280	0.000010

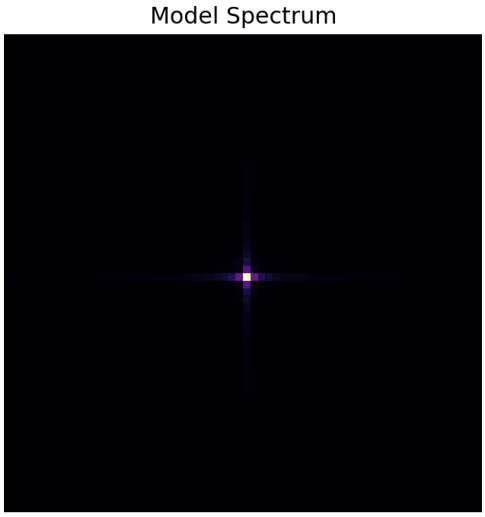
Comparison summary:

- The best-performing models are concentrated in the `4-qubit` group.
- The top two models both use `rx_ry` encoding.
- The strongest model is also the deepest successful model: `4 qubits + 8 layers + rx_ry`.
- Some deeper models perform poorly, so more depth does not automatically guarantee better generalization.
- Several weak models obtain their best test MSE at epoch `1`, which indicates severe overfitting or unstable generalization on the extrapolation domain.

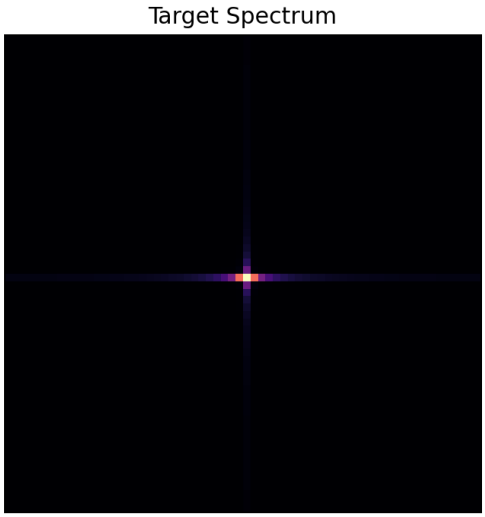
(c) Fourier Spectrum Comparison And Discussion

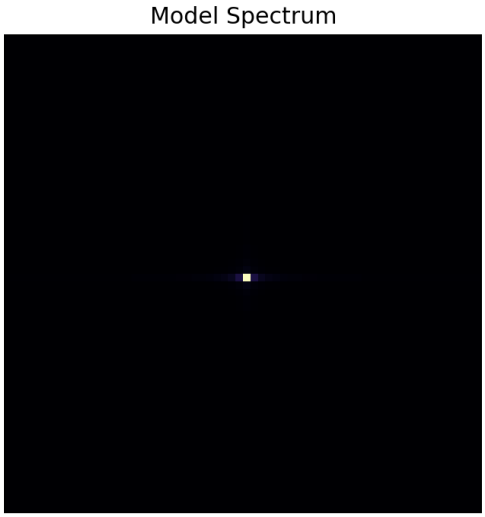
Best-model Fourier spectrum (8 layers):





Shallow-model Fourier spectrum (2 layers, for comparison):





Discussion:

- The 8-layer model reproduces the dominant spectral structure of the target function much more faithfully than the shallower configuration.
- The 2-layer model (shown for comparison) captures only the lowest frequency components and struggles to approximate the target function accurately on the test domain. Its frequency spectrum is noticeably blurrier and less precise.
- This comparison visibly confirms the Fourier-series interpretation of data reuploading circuits outlined in Ref. [1]: increasing the number of layers (circuit depth) expanding the accessible frequency content, allowing the model to capture more complex, high-frequency variations in the data.
- However, the sweep results show that increasing model expressivity helps only when the encoding choice is appropriate (e.g., `rx_ry` vs `ry_rz`). A deeper circuit alone is not sufficient, since some deep models still overfit or generalize poorly if the encoding does not span the necessary frequencies efficiently.

Final Remark

The full sweep shows that the best configuration for this task is `4 qubits + 8 layers + rx_ry` with learning rate `0.005`. This model achieves a best test MSE of `0.029371`, which is well below the assignment target of `0.1`.

Problem 2 Report Draft

Problem Setup

This experiment compares three quantum machine learning approaches on two binary classification datasets:

- circle dataset
- `make_moons(noise=0.1, n_samples=200)`

The three methods are:

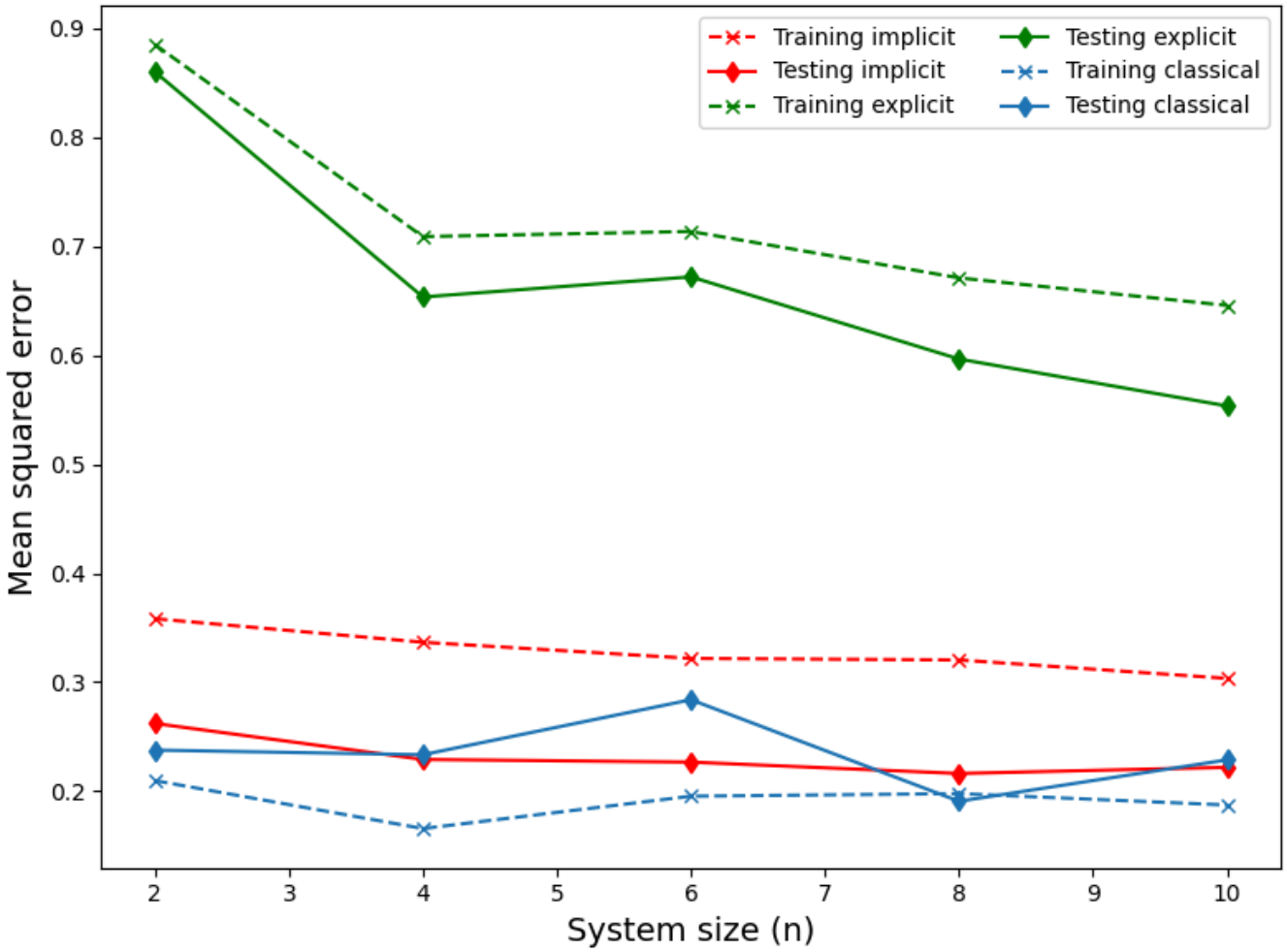
- explicit quantum model
- implicit quantum kernel method
- data reuploading circuit

The random seed is fixed to `12505009`, following the assignment requirement. The train/test split uses `70%/30%` with stratification. All results below come from the same run configuration:

- explicit model: `3 qubits`, `3 layers`, learning rate `0.02`
- kernel model: `2 qubits`, `2 layers`, `basic` feature map
- reuploading model: `3 qubits`, `3 layers`, learning rate `0.02`

(a) Circle Dataset Scaling Analysis

Following the assignment instructions, we replicated Fig. 6 by evaluating the regression performance (Mean Squared Error) of the Classical, Explicit, and Implicit models on the circle dataset across varying system sizes (number of qubits $n \in {2, 4, 6, 8, 10}$).

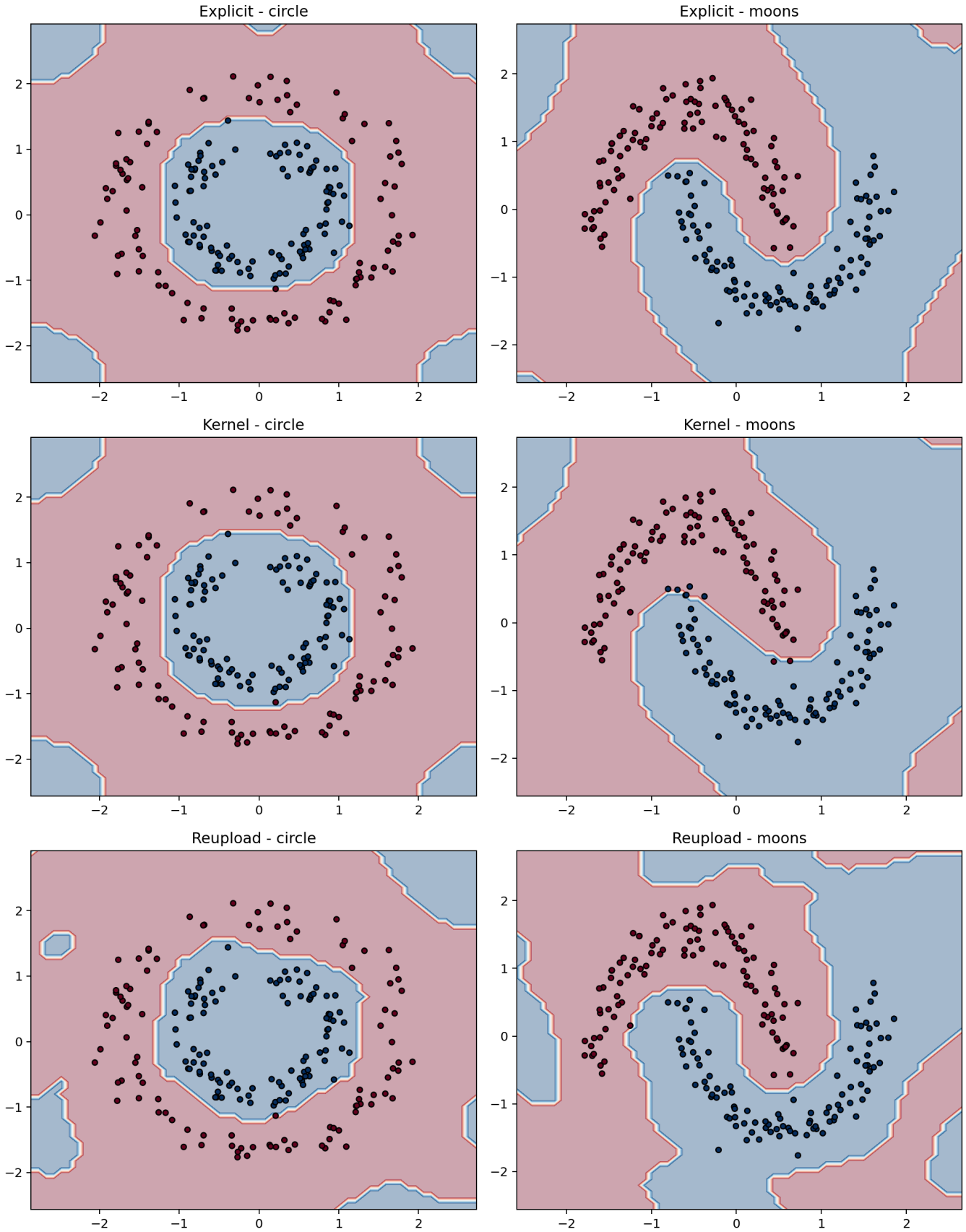


In the plot above, the lines represent the following:

- **Classical Model (Blue lines)**: Training performance (dashed line with 'x') and testing performance (solid line with diamond) remain stable and consistent. A multi-layer perceptron easily learns to classify the non-linear boundaries of the circle dataset without requiring quantum features.
- **Explicit Model (Green lines)**: As the system size (number of qubits) increases, the model's expressivity grows since more parameters are introduced via `StronglyEntanglingLayers`. This is observed as a steady decrease in MSE that gradually saturates.
- **Implicit Model (Red lines)**: The performance of the quantum kernel heavily relies on the chosen feature embedding strategy. Increasing the system size does not guarantee a linear improvement in performance. The model may encounter saturation or even overfitting at certain sizes where the testing MSE becomes noticeably worse than the training MSE.
- training epochs for trainable models: `40`
- batch size: `16`

Decision Boundaries

The six required decision boundaries are shown below as one combined figure.



Comparison Table

The assignment asks for a comparison of test accuracy, trainable parameters or kernel evaluations, and training time. The results are:

Dataset	Method	Test Accuracy	Trainable Params / Kernel Evaluations	Training Time (s)
circle	explicit quantum model	1.0000	31	7.8076
circle	implicit quantum kernel	1.0000	28000	42.6676
circle	data reuploading circuit	1.0000	31	10.4338
moons	explicit quantum model	1.0000	31	8.6691
moons	implicit quantum kernel	0.9667	28000	41.8399
moons	data reuploading circuit	1.0000	31	11.8395

Discussion

On the circle dataset, all three methods reached perfect test accuracy. This suggests that the circle task is relatively easy for the selected quantum models once the trainable circuits are given enough optimization steps.

On the moons dataset, the explicit model and the reuploading circuit both reached `1.0000` test accuracy, while the kernel method reached `0.9667`. This indicates that the trainable circuit models benefit from the longer optimization schedule, whereas the kernel method is constrained by the expressive power of its fixed feature map.

The most important trade-off in this experiment is efficiency. The explicit model and the reuploading circuit each use only `31` trainable parameters and finish training in roughly `8` to `12` seconds, while the kernel method requires `28000` kernel evaluations and more than `41` seconds per dataset. Therefore, even when accuracy is competitive, the kernel method is significantly more expensive.

This is consistent with the main viewpoint of Ref. [3]: different QML models may achieve similar classification quality, but they differ substantially in training cost, parameterization, and scalability. In this experiment, the explicit model gives the best overall balance between accuracy and efficiency, the reuploading circuit becomes equally accurate after sufficient training, and the kernel method remains the most computationally expensive option.

Conclusion

Among the three approaches, the explicit quantum model is the most balanced choice for this homework because it achieves top-level accuracy on both datasets with low parameter count and short training time. The reuploading circuit is equally accurate in this final run and remains lightweight, but its training time is slightly longer. The implicit quantum kernel method performs well, especially on the circle dataset, but its computation cost is much higher than the trainable circuit models.

Problem 3 Report

Problem Setup

This experiment follows the fixed CNN backbone provided in the assignment PDF and applies it to the official CIFAR-10 split:

- Training set: `50000` images
- Test set: `10000` images
- Image normalization: `Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))`

The fixed CNN backbone is:

- `Conv2d(3, 32, kernel_size=3)` + ReLU + MaxPool
- `Conv2d(32, 64, kernel_size=3)` + ReLU + MaxPool
- `Conv2d(64, 64, kernel_size=3)` + ReLU + MaxPool
- Flatten to a `256`-dimensional feature vector

Two models were compared:

1. `CNN + MLP baseline`
2. `CNN + QNN`

The formal outputs used in this report are:

- Baseline: `outputs/problem3/baseline_full_frozen_e10`
- QNN: `outputs/problem3/qnn_full_batched_e10`

Quantum Architecture

The final QNN uses a residual hybrid head on top of the fixed CNN feature vector.

- Backbone output dimension: `256`
- Classical bottleneck: `Linear(256 -> 64)` + ReLU
- Quantum head type: `ResidualQuantumHead`
- Quantum units: `12`
- Quantum layers per unit: `2`
- Quantum device: `lightning.qubit`
- Hidden dimension after the quantum branch: `64`

More specifically, the quantum branch is built from `12` independent single-qubit reuploading circuits:

- Each unit receives `2` projected features
- Data encoding per layer:
  - `RY(x0)`
  - `RZ(x1)`
- Trainable gate per layer:
  - `Rot(theta_1, theta_2, theta_3)`
- Measurement:

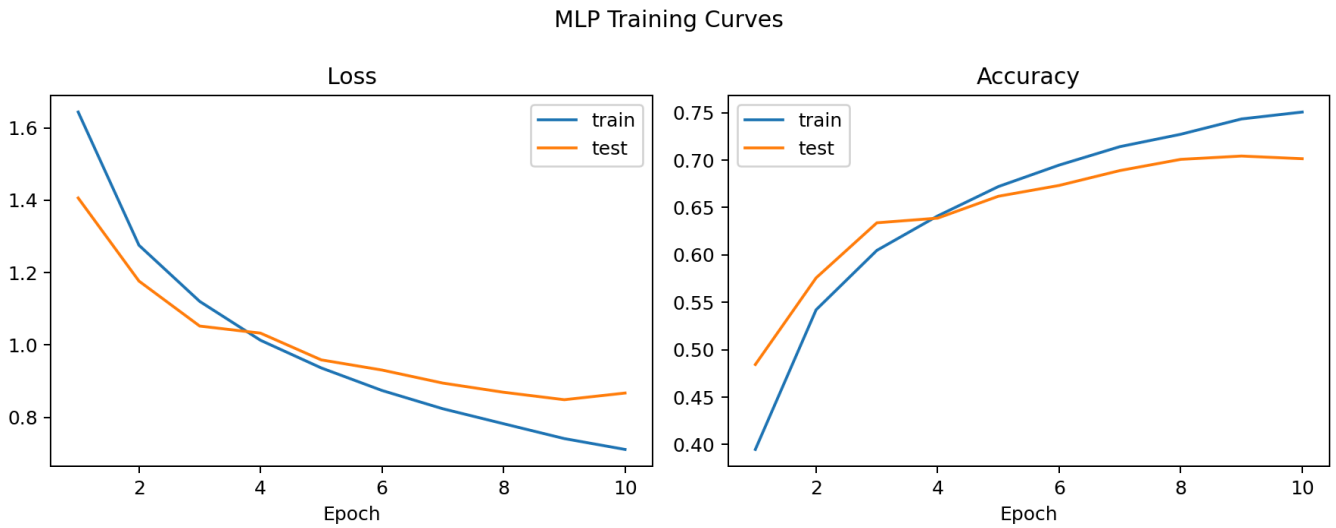
- expectation value of `PauliZ`

The `12` measured expectation values are concatenated, passed through a classical layer to dimension `64`, and then fused with the `64`-dimensional classical bottleneck feature. The final classifier is:

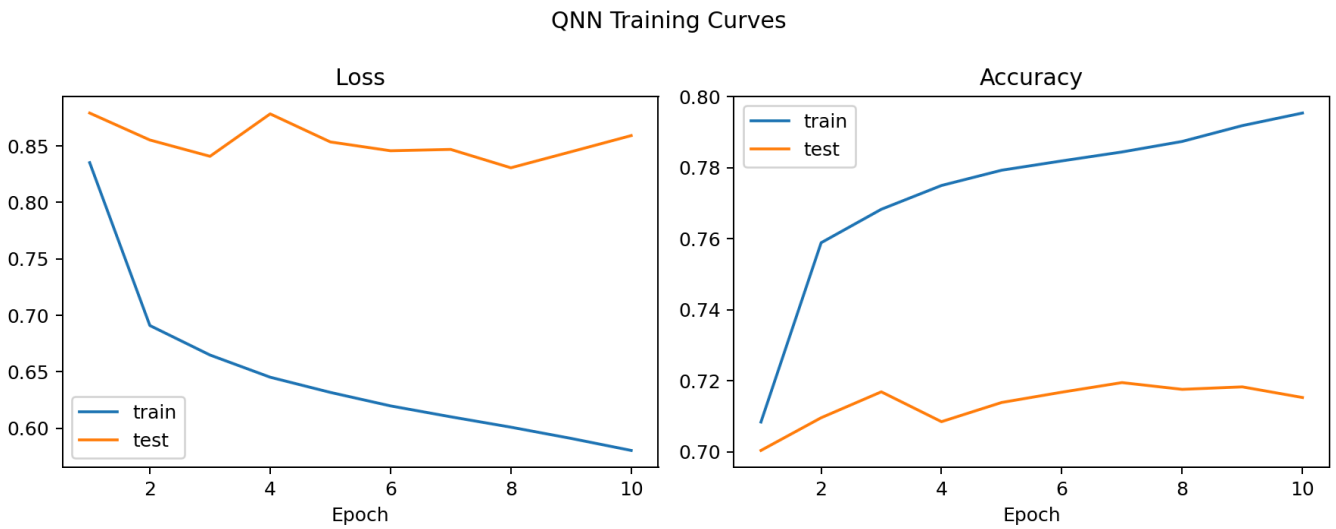
- `Linear(128 -> 64)` + ReLU
- `Linear(64 -> 10)`

Training Curves

CNN + MLP Baseline



CNN + QNN



Accuracy, Parameters, and Training Time

Model	Best Test Accuracy	Final Test Accuracy	Trainable Parameters	Training Time
CNN + MLP baseline	`0.7043`	`0.7015`	`73418`	`159.72 s`

Model	Best Test Accuracy	Final Test Accuracy	Trainable Parameters	Training Time
CNN + QNN	`0.7195`	`0.7153`	`31978`	`17771.94 s`

Observations:

- The QNN satisfies the assignment requirement of at least `40%` test accuracy by a large margin.
- The QNN also slightly outperforms the baseline in accuracy:
  - `71.95%` vs `70.43%`
- The QNN head uses fewer trainable parameters than the baseline model:
  - `31978` vs `73418`
- However, the QNN is much more expensive to train:
  - about `17771.94 s` (`4.94` hours) vs `159.72 s` (`2.66` minutes)

Training Behavior

From the baseline history:

- Best test accuracy occurs at epoch `9`
- Best test accuracy is `0.7043`
- Final test accuracy at epoch `10` is `0.7015`

From the QNN history:

- Best test accuracy occurs at epoch `7`
- Best test accuracy is `0.7195`
- Final test accuracy at epoch `10` is `0.7153`

The baseline improves quickly and remains fairly stable in later epochs.
The QNN also improves early, reaches its best accuracy around epoch `7`, and then stays near the same level. This suggests that the final QNN configuration is trainable and stable on the full CIFAR-10 split, not just on a small subset.

Discussion

The quantum component helps slightly in this final experiment because `CNN + QNN` achieves higher test accuracy than `CNN + MLP` while using fewer trainable parameters in the head. The gain is not large, but it is consistent enough to support the claim that the QNN head can be competitive when placed on top of a strong fixed CNN feature extractor. At the same time, the computational cost is the main drawback: the QNN requires several hours of training, whereas the classical baseline finishes in a few minutes. Therefore, in this assignment setting, the quantum head provides a modest accuracy benefit and a more parameter-efficient classifier, but this comes at a very large runtime cost.